BUG BOUNTY FIELD GUIDE

File-Based Reconnaissance

Extracting URLs, Tokens, API Keys & Secrets from JSON, PDF, XML, DOCX, DOC & XLS Files for Bug Bounty Hunters

- Document OSINT & Forensics
- 2025 Edition

DOCUMENT ANALYSIS & METADATA EXTRACTION

Table of Contents

1. Introduction to File-Based Recon

2. Finding Target Files

2.1 Google Dorks for File Discovery

2.2 Directory Listing & Crawler Methods

3. JSON File Analysis

4. PDF File Analysis

5. XML File Analysis

6. DOCX / DOC File Analysis

7. XLS / XLSX File Analysis

8. Universal Extraction Patterns

9. Tools & Automation

10. Tips, Tricks & Gotchas

11. Best Practices & Legal

1. Introduction to File-Based Recon

Organizations inadvertently leak sensitive information through documents uploaded to websites, shared via cloud storage, or cached by search engines. JSON configuration files, PDF reports, XML data exports, Word documents, and Excel spreadsheets often contain URLs, API keys, authentication tokens, internal network paths, and personal information that attackers can exploit. For bug bounty hunters, file-based reconnaissance is a high-ROI passive technique that requires minimal tooling but yields critical findings. 📁

Unlike JavaScript files that require dynamic analysis, document-based reconnaissance focuses on **static file extraction**. These files are often indexed by Google, exposed in open directory listings, embedded in archived email threads, or shared via public cloud links. The key advantage is that documents often preserve **metadata** — author names, creation software, revision history, internal paths, and embedded links that reveal the organization's infrastructure. 🔍

Why File-Based Recon Matters

Metadata Goldmine: Documents contain author names, software versions, internal paths, and creation dates. **Hardcoded Secrets:** Config exports, API documentation, and integration guides embed keys and tokens. **Hidden URLs:** Embedded hyperlinks in PDFs and Office docs point to internal portals and APIs. **PII Exposure:** Spreadsheets and reports contain employee data, customer records, and financial information. **Historical Data:** Old documents reveal deprecated endpoints and legacy authentication mechanisms. **Passive & Scalable:** No active scanning required — just download and grep.

Scope & Ethics Warning

Downloading publicly accessible files is generally legal — the organization published them. However, **downloading files from authenticated areas, brute-forcing file names, or exploiting path traversal to access restricted documents is not passive** and may violate laws. Only report findings to authorized bug bounty programs. Never distribute leaked PII. Follow responsible disclosure timelines.

2. Finding Target Files

2.1 Google Dorks for File Discovery

GOOGLE DORKS

Google indexes millions of documents daily. Using targeted dorks, you can surface files belonging to your target organization across all supported formats.

DORK **JSON Files**

Configuration files, API exports, and data dumps in JSON format.

```
site:target.com filetype:json ("api_key" OR "token" OR "secret" OR "password" OR "endpoint" OR "url")
```

DORK **PDF Documents**

Reports, manuals, API docs, and internal memos in PDF format.

```
site:target.com filetype:pdf ("confidential" OR "internal" OR "API" OR "endpoint" OR "token" OR "password")
```

DORK **XML Files**

Configuration exports, sitemaps, data feeds, and API responses in XML.

```
site:target.com filetype:xml ("password" OR "token" OR "api" OR "url" OR "endpoint" OR "secret")
```

DORK **DOCX / DOC Files**

Word documents containing procedures, credentials, or integration guides.

```
site:target.com (filetype:docx OR filetype:doc) ("password" OR "token" OR "API" OR "endpoint" OR "secret" OR "internal")
```

DORK **XLS / XLSX Files**

Spreadsheets with user data, inventory, credentials, or API endpoints.

```
site:target.com (filetype:xls OR filetype:xlsx) ("password" OR "token" OR "API" OR "email" OR "phone" OR "SSN")
```

DORK **All Files Combined**

Broad search across all document formats for sensitive keywords.

```
site:target.com (filetype:json OR filetype:pdf OR filetype:xml OR filetype:docx OR filetype:xlsx) ("api_key" OR "token" OR "secret" OR "password")
```

DORK **Directory Listings with Files**

Open directory indexes showing collections of documents.

```
site:target.com intitle:"Index of" (".json" OR ".pdf" OR ".xml" OR ".docx" OR ".xlsx")
```

DORK **Backup & Export Files**

Database exports, backups, and archive files containing documents.

```
site:target.com (filetype:zip OR filetype:tar.gz OR filetype:rar) ("backup" OR "export" OR "data" OR "dump")
```

2.2 Directory Listing & Crawler Methods

DISCOVERY

ALTERNATIVE DISCOVERY METHODS

```
# Wayback Machine for historical files
curl                    -s                    "http://web.archive.org/cdx/search/cdx?
url=*.target.com/*&output=json&fl=original" | \
    grep -iE '\.(json|pdf|xml|docx|xlsx)("|$)' | jq -r '.[]' | sort -u

# gau (GetAllUrls) + filter
gau target.com | grep -iE '\.(json|pdf|xml|docx|doc|xlsx|xls)(\?.*)?$' | sort -u

# Katana crawler with file extraction
katana -u https://target.com -o crawl.txt
grep -iE '\.(json|pdf|xml|docx|xlsx)$' crawl.txt | sort -u

# ffuf for common file names
ffuf -w filenames.txt -u https://target.com/FUZZ \
    -e .json,.pdf,.xml,.docx,.xlsx -mc 200 -o found_files.json

# Common file name wordlist for ffuf
cat > filenames.txt <<EOF
api/config
config
settings
data
backup/export
users
internal/api
report
EOF

# Nuclei for exposed document panels
nuclei -u https://target.com -t http/exposures/files/

# Subfinder + gau combo for file discovery across subdomains
subfinder -d target.com -silent | \
    xargs -I{} sh -c 'gau {} | grep -iE "\.(json|pdf|xml|docx|xlsx)$"' | sort -u
```

3. JSON File Analysis

JSON files are commonly used for configuration, API responses, and data exports. They are plaintext and easy to parse, making them ideal targets for automated secret extraction. 📄

Common JSON File Locations

- `/api/config.json` — Frontend configuration
- `/config/settings.json` — Application settings
- `/data/export.json` — Data exports
- `/package.json` — Node.js dependencies (sometimes exposed)

- `/swagger.json` — API documentation
- `/manifest.json` — App manifests

JSON EXTRACTION COMMANDS

```
# Pretty-print and inspect
python3 -m json.tool config.json | less

# Extract all keys recursively
python3 -c "
import json, sys
def extract(obj, path=''):
    if isinstance(obj, dict):
        for k, v in obj.items():
            new_path = f'{path}.{k}' if path else k
            print(new_path)
            extract(v, new_path)
    elif isinstance(obj, list):
        for i, item in enumerate(obj):
            extract(item, f'{path}[{i}]')
with open(sys.argv[1]) as f:
    extract(json.load(f))
" config.json | grep -iE '(key|token|secret|password|url|api|endpoint)'

# Extract values for specific keys
python3 -c "
import json, sys
with open(sys.argv[1]) as f:
    d = json.load(f)
    for k in ['api_key','token','secret','base_url','endpoint']:
        if k in d:
            print(f'{k}: {d[k]}')
" config.json

# jq one-liners for key extraction
jq 'paths' config.json | grep -i 'key\|token\|secret\|url'
jq '..' | objects | keys[]' config.json | sort -u | grep -i
'api\|key\|token\|url\|endpoint'

# Extract all string values containing 'http'
jq '.. | strings | select(contains(\"http\"))' config.json

# Find all values that look like tokens (long alphanumeric)
jq '.. | strings | select(length > 20 and test(\"^[A-Za-z0-9_-]+$\"))' config.json
```

AUTOMATED JSON SECRET SCANNER

```
#!/usr/bin/env python3
# json_scanner.py - Scan JSON files for secrets

import json, re, sys, os

PATTERNS = {
    'aws_key': r'AKIA[0-9A-Z]{16}',
    'google_api': r'AIza[0-9A-Za-z_-]{35}',
    'jwt': r'eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*',
    'url': r'https?://[^\s"\'<>]+',
       'token':  r'[aA][pP][iI][_\-]?[tT][oO][kK][eE][nN]\s*[:=]\s*["\']?[a-zA-Z0-9_\-]
{16,}',
    'secret': r'[sS][eE][cC][rR][eE][tT]\s*[:=]\s*["\']?[a-zA-Z0-9_\-]{8,}',
    'bearer': r'Bearer\s+[a-zA-Z0-9_\-\•=]+',
    'password': r'[pP][aA][sS][sS][wW][oO][rR][dD]\s*[:=]\s*["\'][^"\']{3,}',
}

def scan_json(filepath):
    try:
        with open(filepath) as f:
            content = f.read()
        data = json.loads(content)
        text = json.dumps(data)  # Flatten for regex
        findings = {}
        for name, pattern in PATTERNS.items():
            matches = set(re.findall(pattern, text))
            if matches:
                findings[name] = matches
        return findings
    except Exception as e:
        return {'error': str(e)}

if __name__ == '__main__':
    for filepath in sys.argv[1:]:
        print(f"\n[+] Scanning: {filepath}")
        results = scan_json(filepath)
        for category, matches in results.items():
            if category == 'error':
                print(f"  [!] Error: {matches}")
                continue
            print(f"  [{category.upper()}] {len(matches)} matches:")
            for m in list(matches)[:3]:
                print(f"    - {m[:60]}")
```

4. PDF File Analysis

PDFs are complex binary documents, but they often contain embedded hyperlinks, metadata, and extractable text. Internal reports, API documentation, and onboarding materials are

frequently saved as PDFs and uploaded to public sites. 📑

What to Look for in PDFs

- **Embedded URLs** — Clickable links to internal portals, APIs, and admin panels
- **Metadata** — Author name, creation software, internal paths in `/Producer` or `/Creator`
- **Extracted Text** — API keys, credentials, and endpoint URLs in the body text
- **Comments & Annotations** — Reviewer notes with internal discussions
- **Attachments** — Embedded files (other PDFs, spreadsheets, configs)

PDF ANALYSIS COMMANDS

```
# Install tools
pip install pdfplumber PyPDF2 pdfminer.six

# Extract text from PDF
python3 -c "
import pdfplumber
with pdfplumber.open('report.pdf') as pdf:
    for i, page in enumerate(pdf.pages):
        text = page.extract_text()
        if text:
            print(f'--- Page {i+1} ---')
            print(text[:2000])
"

# Extract metadata
python3 -c "
from PyPDF2 import PdfReader
reader = PdfReader('report.pdf')
meta = reader.metadata
for k, v in meta.items():
    print(f'{k}: {v}')
"

# Extract URLs from PDF
python3 -c "
import pdfplumber, re
urls = set()
with pdfplumber.open('report.pdf') as pdf:
    for page in pdf.pages:
        text = page.extract_text() or ''
        urls.update(re.findall(r'https?://[^\s<>\"\']+', text))
        # Also check annotations
        for ann in page.annettes or []:
            if 'uri' in ann:
                urls.add(ann['uri'])
print('\n'.join(sorted(urls)))
"

# Search for secrets in extracted text
python3 -c "
import pdfplumber, re
patterns = {
    'aws': r'AKIA[0-9A-Z]{16}',
    'google': r'AIza[0-9A-Za-z_-]{35}',
    'jwt': r'eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*',
    'url': r'https?://[^\s<>\"\']+',
}
with pdfplumber.open('report.pdf') as pdf:
    text = ' '.join([p.extract_text() or '' for p in pdf.pages])
    for name, pat in patterns.items():
        matches = set(re.findall(pat, text))
```

> **PDF Metadata Goldmine**
>
> PDF metadata fields like `/Producer`, `/Creator`, and `/Author` often reveal internal usernames, software versions, and network paths. Example: `Creator: Microsoft Word 16.0 (\DC01\IT\Templates\report.dotx)` reveals the internal domain controller name and file share path.

5. XML File Analysis

XML files are used for configuration (web.config, pom.xml), data exchange (SOAP responses, RSS feeds), and API documentation (WSDL, Swagger in XML format). They are plaintext and highly structured, making them easy to parse for secrets. 📋

Common XML File Types

- `web.config` — IIS configuration with connection strings
- `pom.xml` — Maven build files with repository URLs
- `sitemap.xml` — URL listings including hidden paths
- `WSDL` — SOAP API definitions with endpoint URLs
- `AndroidManifest.xml` — Mobile app configs with API keys
- `.csproj / .sln` — Visual Studio project files with references

XML EXTRACTION COMMANDS

```
# Pretty-print XML
xmllint --format config.xml | less

# Extract all text content
cat config.xml | grep -oP '(?⟺)[^<]+(?=<)' | grep -v '^\s*$'

# Search for specific tags
grep -iE '<(password|connectionstring|apikey|token|secret|endpoint|url|host)>'
config.xml

# XPath extraction with xmllint
xmllint --xpath "//password/text()" config.xml 2>/dev/null
xmllint --xpath "//connectionString/text()" web.config 2>/dev/null
xmllint --xpath "//endpoint/@address" wsdl.xml 2>/dev/null
xmllint --xpath "//url/text()" sitemap.xml 2>/dev/null

# Python XML parsing
python3 -c "
import xml.etree.ElementTree as ET, sys
tree = ET.parse(sys.argv[1])
for elem in tree.iter():
              if  elem.text  and  any(k  in  elem.tag.lower()  for  k  in
['pass','key','token','secret','url','endpoint','host']):
        print(f'{elem.tag}: {elem.text.strip()[:80]}')
" config.xml

# Extract all URLs from XML
python3 -c "
import xml.etree.ElementTree as ET, re
with open('data.xml') as f:
    content = f.read()
urls = set(re.findall(r'https?://[^\s<\"\'\']+', content))
for u in sorted(urls):
    print(u)
"

# AndroidManifest.xml API key extraction
grep -oP 'android:value="[^"]+"' AndroidManifest.xml | grep -iE
'(key|token|secret|api)'
```

6. DOCX / DOC File Analysis

Microsoft Word documents (DOCX and legacy DOC) are ZIP archives containing XML files, embedded images, and metadata. They frequently contain internal procedures with screenshots of admin panels, embedded hyperlinks to internal systems, and revision history with deleted sensitive content. 📝

DOCX Structure (ZIP Archive)

- `word/document.xml` — Main document content
- `word/_rels/document.xml.rels` — External links and embedded objects
- `docProps/core.xml` — Metadata (author, company, last modified)
- `word/comments.xml` — Reviewer comments
- `word/embeddings/` — Embedded files (spreadsheets, other docs)
- `word/media/` — Images (screenshots of internal tools!)

DOCX ANALYSIS COMMANDS

```
# DOCX is a ZIP — extract it first
mkdir docx_extracted
unzip document.docx -d docx_extracted/

# Extract text content
cat docx_extracted/word/document.xml | sed 's/<[^>]*>//g' | tr -s ' \n' | less

# Extract all hyperlinks
cat  docx_extracted/word/_rels/document.xml.rels  |  grep  -oP  'Target="[^"]+"'  |  sed
's/Target=//g;s/"//g'

# Extract metadata (author, company, etc.)
cat docx_extracted/docProps/core.xml | sed 's/<[^>]*>/\n/g' | grep -v '^$'

# Search for secrets in all XML parts
find       docx_extracted/       -name       "*.xml"       -exec       grep       -iH
'password\|token\|api_key\|secret\|endpoint\|url' {} \;

# Python-based extraction with python-docx
pip install python-docx

python3 -c "
from docx import Document
import re

doc = Document('document.docx')

# Extract all text
full_text = '\n'.join([p.text for p in doc.paragraphs])
print('═══ DOCUMENT TEXT ═══')
print(full_text[:3000])

# Extract all hyperlinks
for rel in doc.part.rels.values():
    if 'hyperlink' in rel.reltype:
        print(f'LINK: {rel.target_ref}')

# Extract metadata
props = doc.core_properties
print(f'Author: {props.author}')
print(f'Company: {props.company}')
print(f'Last Modified By: {props.last_modified_by}')

# Search for URL patterns in text
urls = set(re.findall(r'https?://[^\s<\">\']+', full_text))
print(f'\nURLs found: {len(urls)}')
for u in urls:
    print(f'  {u}')
"

# Extract embedded images (may contain screenshots of internal tools)
```

DOCX Revision History Trick

Word documents can contain **track changes** with deleted content still embedded. A reviewer may delete a password or token, but it remains in the XML. Use `cat word/document.xml | grep -iE 'delText|deleted'` or open the document with "Show Markup" enabled to find redacted content.

7. XLS / XLSX File Analysis

Excel spreadsheets are treasure troves for bug bounty hunters. They frequently contain user databases, API endpoint listings, password lists, inventory records, and financial data. Like DOCX, XLSX files are ZIP archives of XML files. 📊

XLSX Structure (ZIP Archive)

- `xl/worksheets/sheet1.xml` — Cell data
- `xl/sharedStrings.xml` — All unique text strings (grep-friendly!)
- `xl/_rels/workbook.xml.rels` — External links
- `docProps/core.xml` — Metadata
- `xl/externalLinks/` — Linked data sources

XLSX ANALYSIS COMMANDS

```
# XLSX is a ZIP — extract it
mkdir xlsx_extracted
unzip data.xlsx -d xlsx_extracted/

# sharedStrings.xml contains ALL unique text — grep goldmine!
cat xlsx_extracted/xl/sharedStrings.xml | sed 's/<[^>]*>/\n/g' | grep -v '^$' | less

# Search sharedStrings for secrets
cat xlsx_extracted/xl/sharedStrings.xml | sed 's/<[^>]*>/\n/g' | \
    grep -iE '(password|token|api_key|secret|endpoint|url|email|phone|ssn)'

# Extract all URLs from shared strings
cat xlsx_extracted/xl/sharedStrings.xml | grep -oP 'https?://[^\s<\"\'\']+' | sort -u

# Extract metadata
cat xlsx_extracted/docProps/core.xml | sed 's/<[^>]*>/\n/g' | grep -v '^$'

# Extract external links (data sources, APIs)
cat xlsx_extracted/xl/externalLinks/_rels/externalLink1.xml.rels 2>/dev/null | \
    grep -oP 'Target="[^"]+"' | sed 's/Target=//g;s/"//g'

# Python extraction with openpyxl
pip install openpyxl

python3 -c "
import openpyxl, re
wb = openpyxl.load_workbook('data.xlsx')

# List all sheet names
print('Sheets:', wb.sheetnames)

# Extract all text from all sheets
for sheet_name in wb.sheetnames:
    sheet = wb[sheet_name]
    print(f'\n=== {sheet_name} ===')
    for row in sheet.iter_rows():
        for cell in row:
            if cell.value and isinstance(cell.value, str):
                # Search for keywords
                                    if  any(k  in  cell.value.lower()  for  k  in
['password','token','api','key','secret','url','endpoint','email']):
                    print(f'[{cell.coordinate}] {cell.value[:100]}')

# Extract all hyperlinks
for sheet in wb.worksheets:
    for row in sheet.iter_rows():
        for cell in row:
            if cell.hyperlink:
                print(f'HYPERLINK: {cell.hyperlink.target}')
"
```

> **The sharedStrings.xml Trick**
>
> All unique text strings in an XLSX file are stored in `xl/sharedStrings.xml`. This single file contains every cell value. Running `grep` on this file is the fastest way to find secrets, emails, URLs, and keywords across the entire spreadsheet without parsing individual sheets.

8. Universal Extraction Patterns

Regardless of file type, certain patterns consistently reveal sensitive data. These regex patterns work across JSON, XML, extracted text, and binary strings. 🔤

Table 1 Universal Secret Patterns

Type	Regex	Applies To
AWS Access Key	`AKIA[0-9A-Z]{16}`	All formats
AWS Secret Key	`[A-Za-z0-9/+=]{40}`	All formats
Google API Key	`AIza[0-9A-Za-z_-]{35}`	All formats
Stripe Key	`(pk_|sk_)(live|test)_[0-9a-zA-Z]{24,}`	All formats
Slack Token	`xox[baprs]-[0-9]{10,13}`	All formats
GitHub Token	`gh[pousr]_[A-Za-z0-9_]{36,}`	All formats
JWT Token	`eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*`	All formats
Bearer Token	`Bearer\s+[a-zA-Z0-9_\-\•=]+`	All formats
Basic Auth	`Basic\s+[A-Za-z0-9+⁄=]+`	All formats
URL	`https?://[^\s"'<>]+`	All formats
Email	`[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`	All formats
IP Address	`\b(?:[0-9]{1,3}\.){3}[0-9]{1,3}\b`	All formats
Private Key	`BEGIN\s+(RSA|OPENSSH|EC|DSA)\s+PRIVATE\s+KEY`	All formats

API Path	`/api/[a-zA-Z0-9_\-/]+`	JSON, XML, text
Connection String	`[Ss]erver=.*[Pp]assword=.*`	XML, config
Auth Token (generic)	`[a-zA-Z0-9_-]{32,64}`	All formats

UNIVERSAL GREP COMMAND

```
# Run this against any extracted text file
PATTERN='(AKIA[0-9A-Z]{16}|AIza[0-9A-Za-z_-]{35}|(pk_|sk_)(live|test)_[0-9a-zA-Z]
{24,}|xox[baprs]-[0-9]{10,13}|gh[pousr]_[A-Za-z0-9_]{36,}|eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-
z0-9_-]*\.[A-Za-z0-9_-]*|Bearer\s+[a-zA-Z0-9_\-\•=]+|https?://[^\s"'<>]+|[a-zA-Z0-
9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}|BEGIN\s+(RSA|OPENSSH|EC)\s+PRIVATE\s+KEY)'

grep -oiE "$PATTERN" extracted_text.txt | sort -u

# Multi-file recursive scan
find . -type f \( -name "*.json" -o -name "*.xml" -o -name "*.txt" \) -exec \
    grep -oiH "$PATTERN" {} \; | tee file_secrets.txt

# On binary files (XLSX/DOCX extracted XML)
find extracted/ -name "*.xml" -exec grep -oiH "$PATTERN" {} + | sort -u
```

9. Tools & Automation

Complete File Recon Pipeline

AUTOMATED FILE SCANNER

```
#!/bin/bash
# file_recon.sh - Complete file-based reconnaissance

TARGET="target.com"
OUTPUT="file_recon_${TARGET}"
mkdir -p "$OUTPUT"/{json,pdf,xml,docx,xlsx,extracted}

echo "[+] Phase 1: Finding files via Google dorks..."
# Use goop or manually construct URLs
# goop -t "$TARGET" -s "site:${TARGET} filetype:json" -o "$OUTPUT/json_urls.txt"
# goop -t "$TARGET" -s "site:${TARGET} filetype:pdf" -o "$OUTPUT/pdf_urls.txt"
# goop -t "$TARGET" -s "site:${TARGET} filetype:xml" -o "$OUTPUT/xml_urls.txt"
# goop -t "$TARGET" -s "site:${TARGET} filetype:docx" -o "$OUTPUT/docx_urls.txt"
# goop -t "$TARGET" -s "site:${TARGET} filetype:xlsx" -o "$OUTPUT/xlsx_urls.txt"

# Alternative: gau-based discovery
gau "$TARGET" | grep -iE '\.(json|pdf|xml|docx|xlsx)(\?.*)?$' | sort -u >
"$OUTPUT/all_file_urls.txt"

echo "[+] Phase 2: Downloading files..."
mkdir -p "$OUTPUT/downloads"
cat "$OUTPUT/all_file_urls.txt" | while read url; do
    filename=$(echo "$url" | md5sum | cut -d' ' -f1)
    ext=$(echo "$url" | sed 's/.*\.//;s/?.*//')
        curl -s -L "$url" -o "$OUTPUT/downloads/${filename}.${ext}" --max-time 15
2>/dev/null
    sleep 1
done

echo "[+] Phase 3: Processing JSON files..."
for f in "$OUTPUT/downloads"/*.json; do
    [ -f "$f" ] || continue
    python3 -c "
import json, re, sys
patterns = {
    'url': r'https?://[^\s\"<>]+',
    'aws': r'AKIA[0-9A-Z]{16}',
    'google': r'AIza[0-9A-Za-z_-]{35}',
    'jwt': r'eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*',
    'token': r'[aA][pP][iI][_\-]?[tT][oO][kK][eE][nN].{0,20}[a-zA-Z0-9_\-]{16,}',
}
try:
    with open(sys.argv[1]) as fh:
        text = json.dumps(json.load(fh))
    for name, pat in patterns.items():
        matches = set(re.findall(pat, text))
        if matches:
            print(f'[{name}] {len(matches)} in {sys.argv[1]}')
            for m in list(matches)[:3]:
                print(f'  {m[:60]}')
except: pass
```

Python Multi-Format Analyzer

PYTHON: UNIVERSAL FILE ANALYZER

```
#!/usr/bin/env python3
# file_analyzer.py - Analyze any document for secrets

import re, sys, os, json, zipfile
from pathlib import Path

PATTERNS = {
    'aws_key': r'AKIA[0-9A-Z]{16}',
    'google_api': r'AIza[0-9A-Za-z_-]{35}',
    'stripe_key': r'(pk_|sk_)(live|test)_[0-9a-zA-Z]{24,}',
    'slack_token': r'xox[baprs]-[0-9]{10,13}',
    'github_token': r'gh[pousr]_[A-Za-z0-9_]{36,}',
    'jwt': r'eyJ[A-Za-z0-9_-]*\.eyJ[A-Za-z0-9_-]*\.[A-Za-z0-9_-]*',
    'url': r'https?://[^\s"'<>]+',
    'email': r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}',
    'bearer': r'Bearer\s+[a-zA-Z0-9_\-\•=]+',
    'private_key': r'BEGIN\s+(RSA|OPENSSH|EC|DSA)\s+PRIVATE\s+KEY',
}

def analyze_text(text, source):
    findings = {}
    for name, pattern in PATTERNS.items():
        matches = set(re.findall(pattern, text))
        if matches:
            findings[name] = matches
    return findings

def analyze_json(filepath):
    with open(filepath) as f:
        text = json.dumps(json.load(f))
    return analyze_text(text, filepath)

def analyze_xml(filepath):
    with open(filepath) as f:
        text = f.read()
    return analyze_text(text, filepath)

def analyze_pdf(filepath):
    try:
        import pdfplumber
        with pdfplumber.open(filepath) as pdf:
            text = ' '.join([p.extract_text() or '' for p in pdf.pages])
        return analyze_text(text, filepath)
    except ImportError:
        print("[!] Install pdfplumber: pip install pdfplumber")
        return {}

def analyze_docx(filepath):
    text_parts = []
    links = []
    with zipfile.ZipFile(filepath) as z:
```

10. Tips, Tricks & Gotchas

Pro Tips

Check ZIP contents without extracting. Use `unzip -l` to list contents first. **sharedStrings.xml is the XLSX goldmine.** All unique strings live here — one grep covers the entire spreadsheet. **DOCX images may be screenshots.** Check `word/media/` for internal tool screenshots. **PDF metadata reveals authors and software paths.** Use `pdfinfo` on every PDF. **XML config files contain connection strings.** `web.config` and `app.config` frequently have database passwords. **JSON APIs may expose internal objects.** A missing field filter can leak sensitive fields. **Old file versions matter.** Wayback Machine may have earlier versions with more sensitive data. **File names are clues.** `backup_2023_users.json` tells you exactly what's inside.

Common Gotchas

False positives on template files. Example documents with placeholder text like `password: [YOUR_PASSWORD]` are not valid findings. **Public API keys.** Some keys (e.g., Google Maps client-side keys) are intentionally public and scoped. Verify before reporting. **CORS doesn't make an endpoint exploitable.** Finding a URL in a file doesn't mean it's vulnerable. **Rate limiting on downloads.** Don't hammer servers with bulk downloads. Add delays. **File types can be mislabeled.** A `.json` file might actually be HTML. Check content-type. **Encoding issues.** XLSX/DOCX XML may use special encoding. Use `errors='ignore'` when parsing.

Quick Reference Checklist

Table 2 Per-File-Type Checklist

File Type	What to Check	Key Location
JSON	Keys, nested objects, arrays	All text
PDF	Metadata, links, text, images	pdfinfo, annotations, media
XML	Tag names with sensitive keywords	Tags & attributes
DOCX	Links, comments, images, metadata	_rels/, comments.xml, docProps/
XLSX	Shared strings, external links, formulas	xl/sharedStrings.xml, externalLinks/

11. Best Practices & Legal

Validation Checklist

Table 3 Before Reporting

Step	Action	Why
1	Verify file is publicly accessible (no auth required)	Files from authenticated areas are not valid
2	Confirm secret/key is active (not a placeholder)	Template values are false positives
3	Check if key is intentionally public/scoped	Some client-side keys are expected
4	Verify endpoint URL responds	Dead URLs are not valid findings
5	Document exact file URL and location	Reproducibility for triagers
6	Minimize downloaded file retention	Delete PII after analysis

Report Template

File Exposure Report Template

Title: [P2] Sensitive Data Exposure in Publicly Accessible XLSX File

Description: A publicly accessible Excel spreadsheet on the target domain contains employee email addresses, internal API endpoint URLs, and authentication tokens.

File URL: https://target.com/exports/user_data_2024.xlsx

Discovery Method: Google dork `site:target.com filetype:xlsx email` followed by automated analysis of sharedStrings.xml.

Data Found: 150+ employee email addresses, 3 internal API endpoint URLs (https://internal-api.target.com/v2/), and 1 Bearer token in the "Integration" sheet.

Impact: Employee emails enable spear-phishing. Internal API URLs expose attack surface. The Bearer token may allow unauthorized API access depending on scope.

Remediation: Remove the file from public access. Implement authentication for /exports/ directory. Rotate the exposed Bearer token. Review file upload policies to prevent future leaks.

Legal & Ethical Guidelines

Only download from public URLs. Never use authentication bypass or path traversal. **Delete PII after analysis.** Do not retain leaked personal data. **Do not use discovered credentials.** Report only — never exploit. **Follow responsible disclosure.** Allow time for remediation. **Report to authorized programs only.** Ensure the target has a bug bounty program.

Recommended Resources

- **pdfplumber** — github.com/jsvine/pdfplumber
- **PyPDF2** — github.com/py-pdf/PyPDF2
- **python-docx** — github.com/python-openxml/python-docx
- **openpyxl** — github.com/theorchard/openpyxl
- **gau** — github.com/lc/gau
- **katana** — github.com/projectdiscovery/katana
- **goop** — github.com/six2dez/goop
- **Wayback Machine** — web.archive.org